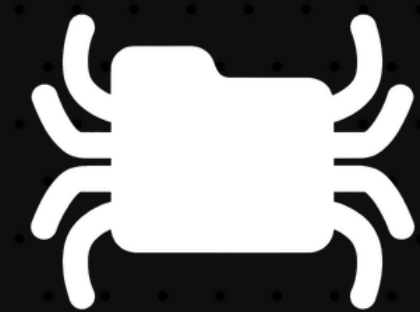
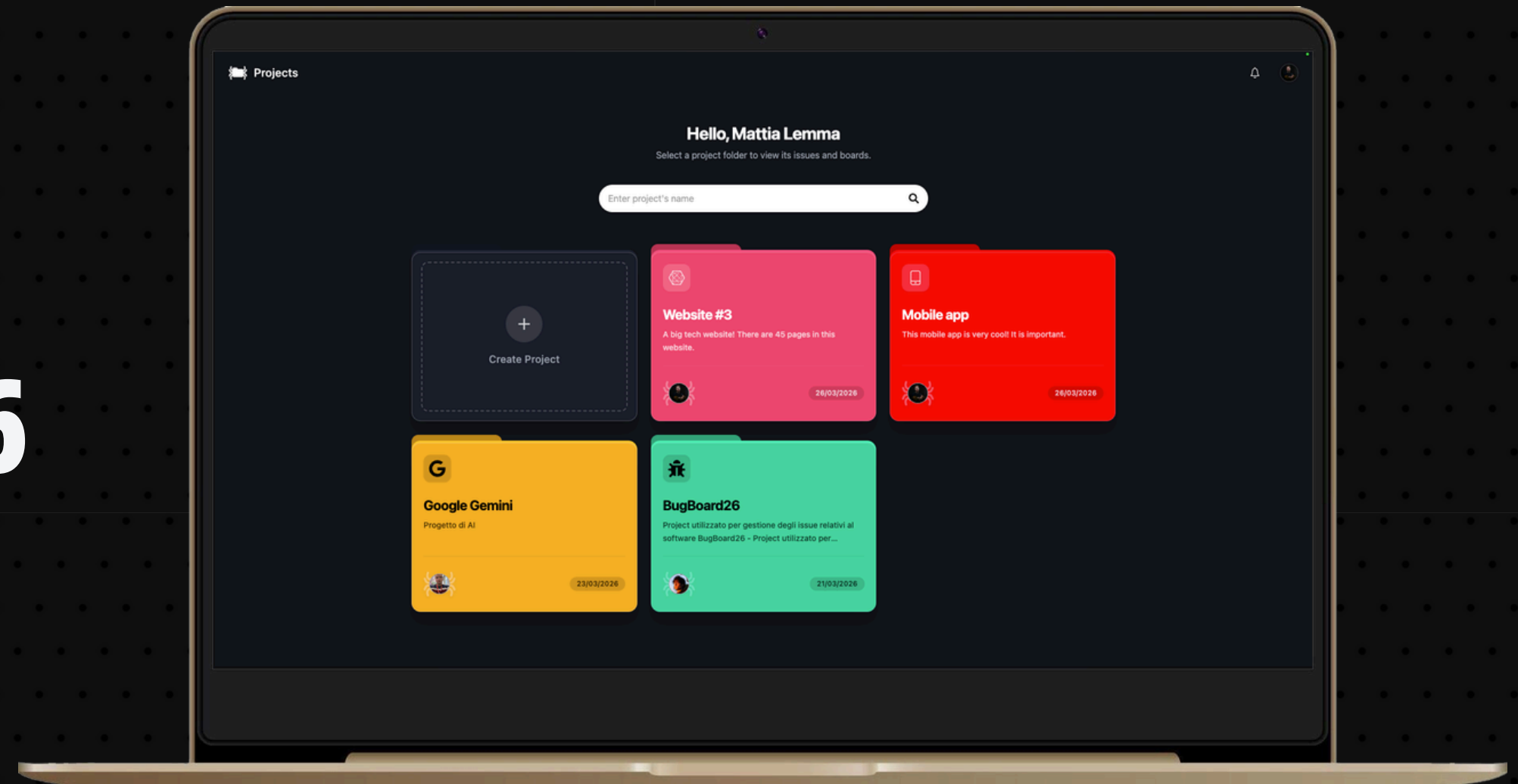


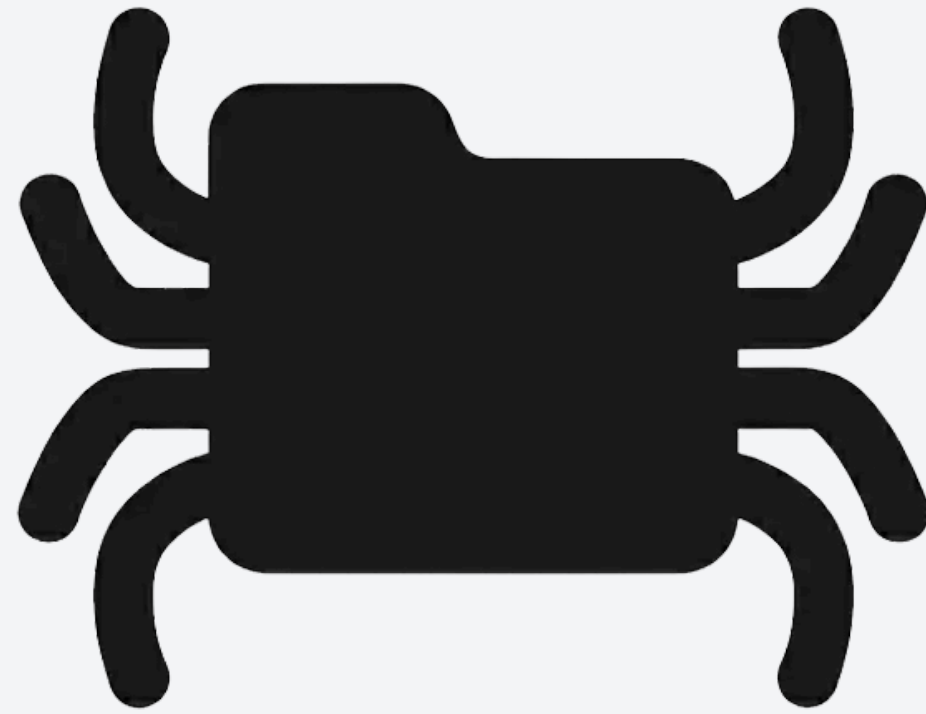


INTRODUZIONE AL PROGETTO SOFTWARE

BugBoard26



Mattia Lemma, Carmine Sgariglia, Massimo Russo
(N86005170) - (N86005069) - (N86005016)



BugBoard26

www.bugboard.it

L'**applicazione web** è progettata per raccogliere, organizzare e monitorare in modo semplice ed efficace bug, segnalazioni, idee e attività di miglioramento. Attraverso un approccio pratico e strutturato, mira a favorire la collaborazione tra i membri del team, aumentare la visibilità sulle criticità e rendere più fluido il processo di gestione e risoluzione delle issue.

Parole chiave

Alcune parole che contraddistinguono BugBoard.



Le fasi

Le macrofasi seguite per la realizzazione del progetto sono coerenti con il modello di sviluppo Waterfall.

- 1. Requirements Engineering**
- 2. System Design**
- 3. Software and UI Design**
- 4. Implementation**
- 5. Testing**
- 6. Verification e Validation**

Requirements Engineering

Nella fase di analisi e **specifica dei requisiti** sono state raccolte le esigenze del progetto, definiti gli obiettivi principali e individuate le funzionalità necessarie. Sono stati inoltre chiariti i vincoli, le priorità e i requisiti tecnici e funzionali, così da disporre di una base chiara e condivisa per le successive fasi di progettazione e sviluppo.

Organizzazione del team

REQUIREMENTS ENGINEERING



È stato **organizzato il team**, definendo i ruoli dei componenti, ed è stato predisposto uno spazio condiviso su Notion per raccogliere proposte, problemi emersi e attività svolte, favorendo il coordinamento e la tracciabilità del lavoro. **Link al Notion:** urly.it/31f8z7

Organizzazione del team

REQUIREMENTS ENGINEERING

(Responsabilità)	(Ruoli)			
		FRONTEND	BACKEND	DATABASE
	PRINCIPALE	Mattia	Carmine	Massimo
	SECONDARIA	Massimo	Massimo	Carmine
	TERZIARIA	Carmine	Mattia	Mattia

Use Case

REQUIREMENTS ENGINEERING



La successiva attività di specifica e rappresentazione dei requisiti è stata supportata dall'impiego di **Visual Paradigm**, attraverso il quale sono stati realizzati gli **Use Case Diagram** di sistema.

SRS

REQUIREMENTS ENGINEERING



L'output conclusivo della fase di engineering è il Software Requirements Specification (**SRS**), redatto integralmente in **LaTeX** per garantire una documentazione tecnica formale, strutturata e coerente, che integra ogni requisito funzionale e vincolo di sistema.

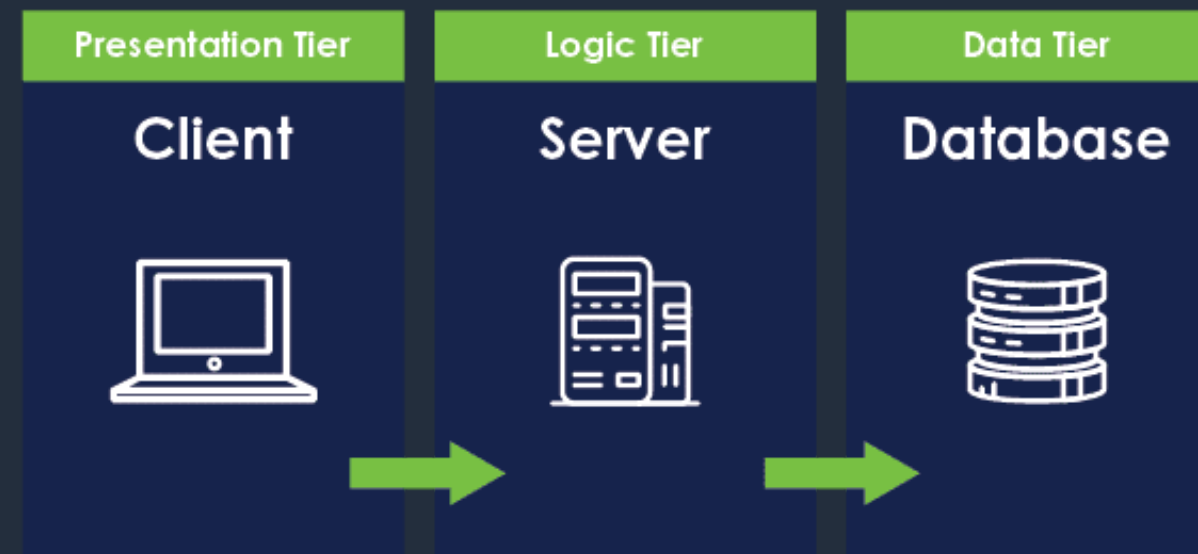
System Design

In questa fase, sulla base dei requisiti definiti nell'SRS, è stata progettata l'architettura generale del sistema, individuando i principali sottosistemi e le relative interazioni. I requisiti sono stati poi distribuiti tra i diversi componenti software, così da ottenere una struttura chiara, modulare e manutenibile. Sono state inoltre valutate le tecnologie e le soluzioni architetture più adatte a supportare le successive fasi di dettaglio e implementazione.

Architettura

SYSTEM DESIGN

Three-tier



La **three-tier** architecture organizza il sistema in tre livelli distinti: interfaccia utente, logica applicativa e database, così da garantire una gestione più ordinata, scalabile e manutenibile dell'intera piattaforma.

Frontend

SYSTEM DESIGN



Sviluppato con **React** e **TypeScript**, utilizzando **Vite** per un ambiente di sviluppo rapido ed efficiente. La navigazione tra le pagine è gestita tramite React Router, mentre TanStack Query supporta la gestione dei dati e delle chiamate asincrone. Lo stile dell'interfaccia è realizzato con Tailwind CSS, per garantire un design moderno, coerente e facilmente mantenibile.

Backend

SYSTEM DESIGN



Realizzato con **Django** e **Django REST Framework**, tecnologie che permettono di sviluppare API robuste, sicure e ben strutturate. L'applicazione segue un'architettura a monolite modulare, in cui le diverse funzionalità sono organizzate in moduli separati ma integrati, così da mantenere il codice ordinato, scalabile e più semplice da gestire.

Database

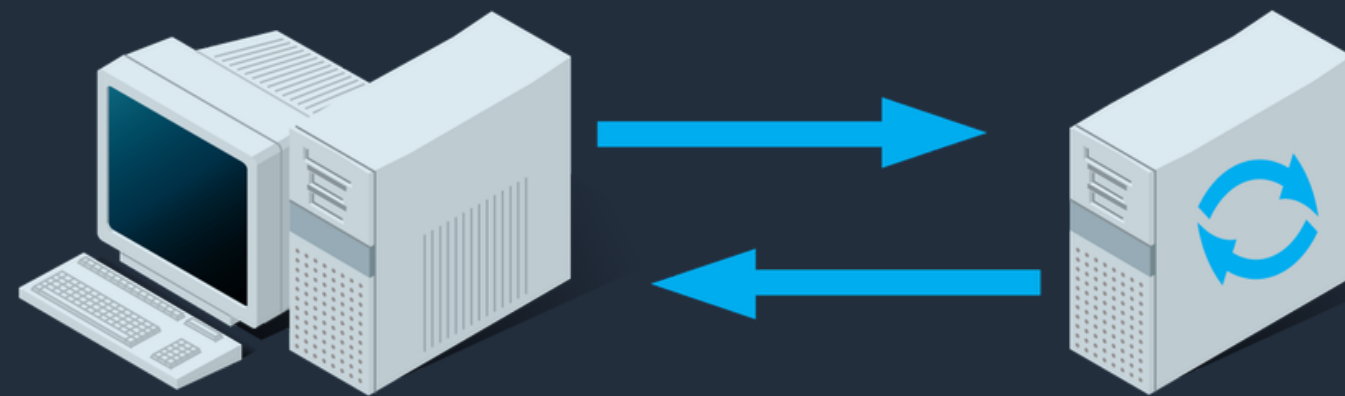
SYSTEM DESIGN



PostgreSQL è stato scelto per la sua affidabilità, stabilità e capacità di gestire dati strutturati in modo efficiente. Permette di organizzare in modo coerente informazioni come utenti, progetti e issue, garantendo buone prestazioni e integrità dei dati.

SSE

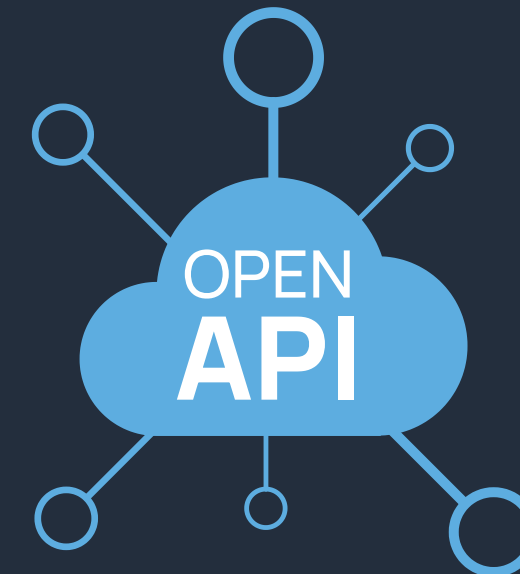
SYSTEM DESIGN



La comunicazione realtime è gestita tramite **Server-Sent Events (SSE)**, una tecnologia che consente al server di inviare aggiornamenti in tempo reale al client. Questo permette di mostrare modifiche e notifiche in modo immediato, rendendo l'interazione più dinamica e reattiva.

API

SYSTEM DESIGN



L'esposizione delle funzionalità applicative avviene tramite **API REST**, progettate con criteri di coerenza contrattuale e documentate attraverso specifiche **OpenAPI**, così da garantire chiarezza, interoperabilità e semplicità di integrazione tra i diversi livelli del sistema.

Deploy

SYSTEM DESIGN



Il deploy e l'infrastruttura si basano su **Docker Compose** e **Nginx**: il primo semplifica la gestione e l'orchestrazione dei servizi dell'applicazione, mentre il secondo funge da web server e reverse proxy, migliorando organizzazione, accessibilità e distribuzione del sistema.

Deploy

SYSTEM DESIGN



Google Cloud



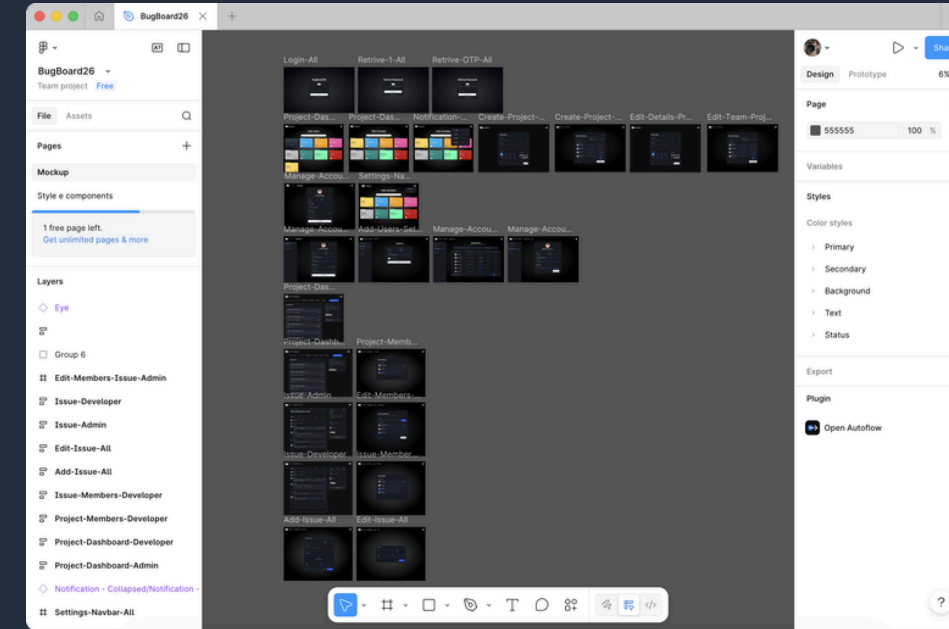
I servizi esterni includono **Google Cloud Storage**, utilizzato per la gestione dei file, e **Brevo**, impiegato come provider email per l'invio di OTP e credenziali, a supporto dei processi di verifica, autenticazione, creazione degli account e mailing.

Software & UI Design

Sono stati definiti i sottosistemi software di dettaglio e l'organizzazione interna dei componenti applicativi, con particolare attenzione alla coerenza tra architettura logica, funzionalità e interazione utente.

Wireframe

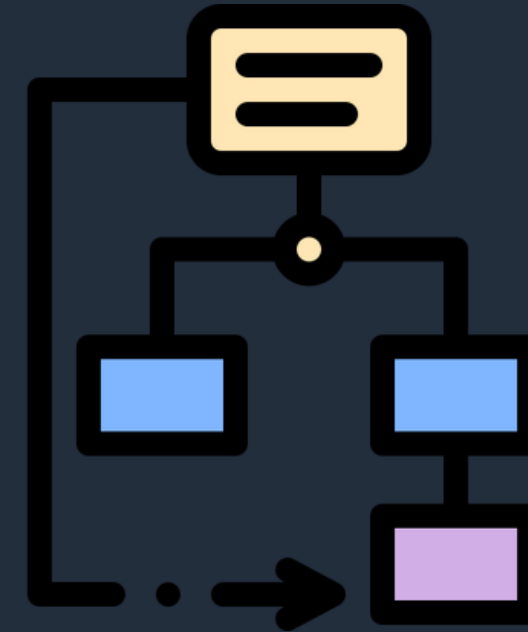
SOFTWARE & UI DESIGN



L'interfaccia grafica del sistema è stata progettata mediante **Figma**, utilizzato per la realizzazione di **wireframe** e prototipi. Questa attività ha permesso di tradurre le scelte progettuali in una struttura più concreta e in un'interfaccia utente coerente, usabile e aderente agli obiettivi del progetto.

Modellazione del sistema

SOFTWARE & UI DESIGN



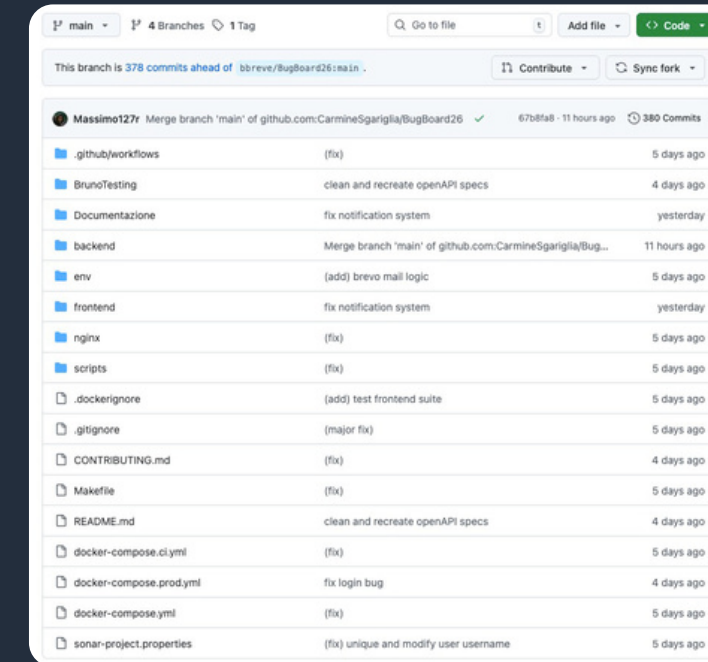
Sono stati utilizzati **UML**, il diagramma delle classi e il **diagramma del database**, strumenti utili per rappresentare in modo chiaro la struttura del sistema, le relazioni tra le componenti software e l'organizzazione dei dati.

Implementation

Sono stati sviluppati e integrati tutti i componenti del sistema, comprendendo la gestione del repository Git per il versionamento e la collaborazione, la configurazione degli ambienti tramite Docker, l'implementazione del frontend e del backend, la progettazione del database, l'integrazione dei servizi di autenticazione, realtime e testing, nonché la predisposizione dei servizi infrastrutturali e cloud necessari al deploy e all'esecuzione dell'applicazione.

GitHub

IMPLEMENTATION



Il progetto è ospitato su **GitHub** in un unico repository che contiene frontend, backend, test e configurazioni di deploy. Lo sviluppo avviene tramite branch dedicati e pull request verso main, mentre le **GitHub Actions** automatizzano test, controlli di qualità e deploy, rendendo il workflow più sicuro, ordinato e continuo.

Dominio

IMPLEMENTATION



La piattaforma è stata collegata correttamente a Register ed è ora online e visibile sui motori di ricerca, pronta per offrire una presenza digitale facilmente raggiungibile dagli utenti.

www.bugboard.it

Testing

Sono stati adottati strumenti differenziati per verificare i diversi livelli del sistema: **Vitest** per il **frontend**, i test **Python** per il backend, **Bruno** per il collaudo delle API REST e **SonarQube** per l'analisi statica della qualità del codice. In questo modo è stato possibile validare il corretto funzionamento delle funzionalità implementate, individuare eventuali anomalie e monitorare la qualità complessiva del software.

Tools

VERIFICATION & VALIDATION



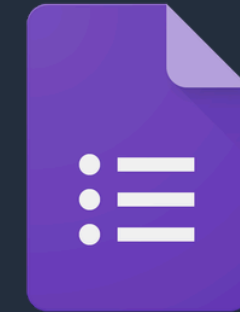
Con **Bruno** sono stati eseguiti test sulle API REST per verificare il corretto funzionamento degli endpoint e la coerenza dei contratti applicativi. Con **SonarQube** è stata invece effettuata un'analisi statica del codice, utile per controllare qualità, manutenibilità e possibili criticità del software.

Verification & Validation

Verifica della correttezza tecnica del sistema attraverso attività di controllo del comportamento del software, accertando la conformità rispetto ai requisiti definiti.

Feedback

VERIFICATION & VALIDATION



Google Forms

Per la validazione lato utente, sono stati predisposti account dedicati per consentire a utenti esterni di accedere alla piattaforma e valutarne l'usabilità. I feedback, raccolti tramite Google Form, sono stati poi sintetizzati in un documento contenente analisi e riepilogo dei risultati.

Euristica

VERIFICATION & VALIDATION



È stata inoltre condotta una valutazione euristica interna, finalizzata ad analizzare l'interfaccia e le principali funzionalità del sistema sulla base di criteri di usabilità riconosciuti. Tale attività ha permesso di formulare una valutazione qualitativa complessiva dell'esperienza d'uso.

Possibili espansioni & cambiamenti

Questa sezione raccoglie alcune possibili evoluzioni del sistema, evidenziando sia le funzionalità che potrebbero essere introdotte in sviluppi futuri, sia alcune scelte progettuali e implementative che avrebbero potuto essere affrontate in modo diverso. L'obiettivo è offrire una riflessione conclusiva sui margini di estensione e sugli aspetti suscettibili di miglioramento.

Espansioni

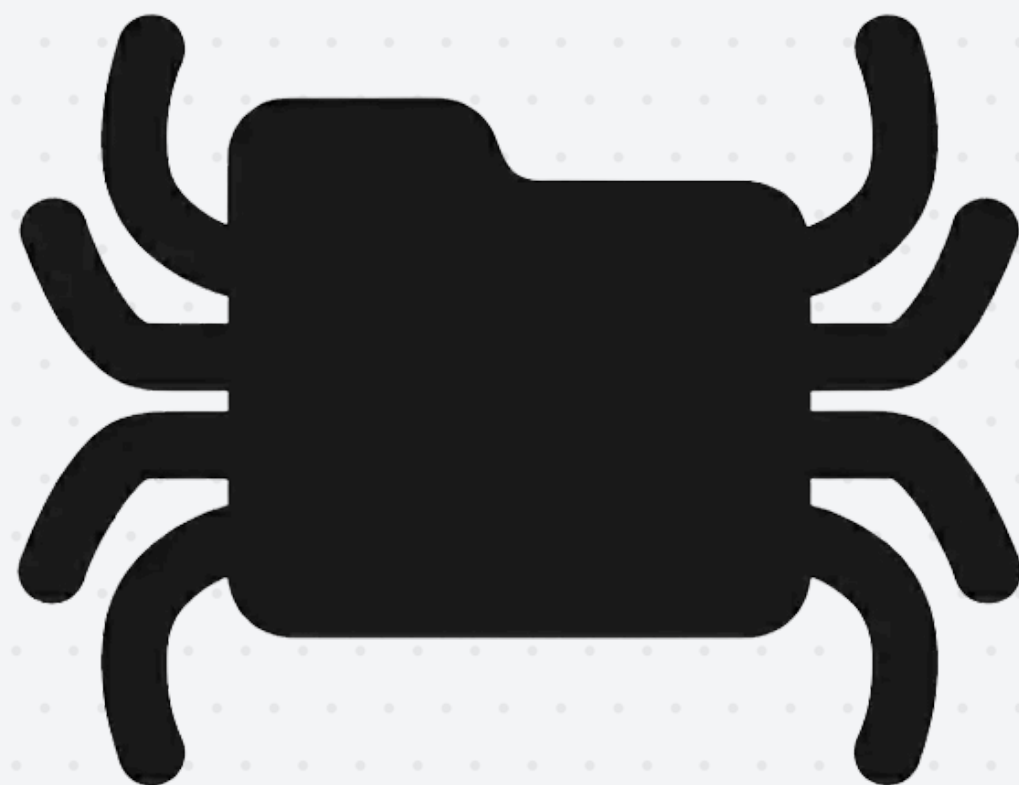
ESPANSIONI & CAMBIAMENTI

Le possibili espansioni future comprendono l'introduzione di **nuove funzionalità** per la gestione dei team e dei workflow, una maggiore personalizzazione di dashboard e notifiche, l'integrazione con strumenti esterni di sviluppo e collaborazione, e il **potenziamento dell'analisi dei dati** tramite report e statistiche. In questo caso, l'obiettivo è ampliare le capacità del sistema e arricchirne gli ambiti di utilizzo.

Cambiamenti

ESPANSIONI & CAMBIAMENTI

Tra i cambiamenti che avrebbero potuto migliorare il software rientrano interventi mirati all'**ottimizzazione delle prestazioni**, al rafforzamento dell'accessibilità e dell'usabilità dell'interfaccia, all'estensione della copertura di testing e al consolidamento dell'infrastruttura di deploy e monitoraggio. Si tratta quindi di miglioramenti rivolti alla qualità, all'affidabilità e alla robustezza complessiva del sistema, senza introdurre necessariamente nuove funzionalità.



BugBoard26

www.bugboard.it